姓名（待补充）

Unity 游戏客户端开发 / Unity 游戏开发工程师

手机（待补充） | 邮箱（待补充） | 所在城市（待补充） | GitHub/作品链接（待补充）

教育背景

学校 / 专业 / 学历 / 毕业时间（待补充）

技术能力

Unity / C#： Unity 2022 LTS、MonoBehaviour 生命周期、UGUI/TMP、Transform、Rigidbody2D/Collider2D、Animator 基础；能将游戏规则与表现层拆分。

游戏系统与网络： 固定 tick 模拟、自定义 2D 碰撞、状态机、对象复用、UDP 输入同步与有限窗口回滚原型、EditMode/PlayMode 测试。

DOTS / 算法： 了解 Entities 1.x、Baker、ISystem、DynamicBuffer、Burst/Job、NativeContainer；能分析 A*、最小堆、KD-Tree KNN 与 ORCA 数据流及性能边界。

项目经历

Pocket Bump｜2D 双人物理竞技游戏 | 独立 Unity 开发

Unity 2022.3 / C# / UGUI + TMP / UDP / 自定义固定步长模拟 / Unity Test Framework

- **项目背景：** 围绕双人争抢弹珠、篮筐蓄力射击与场地得分构建同机/局域网对战；将比赛推进、碰撞和计分从 Unity 组件中抽离为可复制的纯状态模拟。
- **固定 tick 与碰撞：** 以 60 Hz 推进 5 状态比赛流程，使用圆-圆、圆-AABB/OBB 和速度相关子步处理玩家/弹珠/篮筐碰撞；在 LateUpdate 插值前后状态，降低固定帧视觉抖动。
- **回滚与协议：** 针对 UDP 远端输入迟到，设计 2 tick 输入延迟、12 tick 快照环和上一输入预测；检测历史输入差异后恢复并重演，以 1 byte 压缩操作，并通过版本/规则握手、ack 与状态校验和发现连接异常和失同步。
- **工程化与验证：** 复用 1–8 个弹珠对象，避免回合内重复创建/销毁；按 tick 清理回滚后的瞬时特效；编写 29 个 EditMode/PlayMode 用例，覆盖回滚输入带、握手、碰撞/抢夺/计分和菜单流程（本次未在本机重新执行）。

AStarEcs｜DOTS A* 源码研读与工程整理 | 基于 Athomield 既有工程

Entities 1.3 / Burst / Job System / Unity Physics / A* / KD-Tree KNN / ORCA

- **架构梳理：** 围绕 Component/Baker/System 整理网格烘焙、障碍与惩罚扫描、连通岛、请求队列、A* 路径回写和 LocalTransform 跟随的数据流；默认示例网格为 30×30（900 节点），每帧最多处理 25 个请求。
- **算法与性能分析：** 阅读 NativeMinHeap + NativeHashSet 的 A*、KD-Tree 批量 KNN 与 ORCA 速度约束；分析 330 Entity 圆阵示例，并识别同步 Complete、每帧 Native 临时分配及 open set 线性扫描等优化空间。KNN 为第三方 MIT 来源，不作为个人原创实现。

技术内容依据当前项目源码整理；投递前请补齐个人信息、项目时间与作品链接。